

# n8n M-Pesa Mastery

Build Production-Ready STK Push Payment Workflows - No Backend code

# n8n M-Pesa Mastery : Course Handbook

---

## Build Production-Ready STK Push Payment Workflows - No Backend code

This Markdown file is the master draft. Export to PDF via **Pandoc** or **Typora**:

```
pandoc docs/handbook.md -o docs/handbook.pdf --pdf-engine=xelatex --toc -V
geometry:margin=1in
```

---

## Table of Contents

- 0. Welcome
  - 1. Introduction to n8n
  - 2. Understanding the Daraja API
  - 3. OAuth in n8n
  - 4. Building the STK Push Workflow
  - 5. Handling the Callback
  - 6. Status Query & Reconciliation
  - 7. Production Hardening
  - 8. Real-World Use Cases
  - 9. Beyond STK Push
  - 10. Selling Your Skill
  - 11. Appendix: Troubleshooting
- 

## Welcome

---

**What you'll build:** production-ready STK Push payment workflows using n8n, with no custom backend service required. By the end of this course you'll have:

- A self-hosted n8n instance behind HTTPS
- 4 core workflows (auth, STK push, callback, reconciliation)
- A Postgres database tracking every transaction

- 5 real-world use-case templates (e-commerce, SaaS, school fees, donations, WhatsApp)
- The skills to sell this as a service to Kenyan businesses

#### Prerequisites:

- Comfort with the command line
- A credit card for a \$6/month VPS (optional for local dev)
- A Safaricom Developer account (free)
- 6–8 hours of focused time

---

## Module 1 : Introduction to n8n

---

### 1.1 What is n8n?

n8n (pronounced *n-eight-n*) is an open-source workflow automation tool. Think Zapier or Make, but:

- **Self-hostable** — no per-task fees
- **Code-friendly** — drop into JavaScript when you need to
- **400+ integrations** out of the box
- **Fair-code licensed** — free for internal use, paid for embedding in products

### 1.2 Why n8n for M-Pesa?

The traditional approach to M-Pesa integration is a Spring Boot / Node.js / Django service that:

1. Fetches an OAuth token
2. Builds the STK push payload (timestamp + password)
3. POSTs to Daraja
4. Exposes a `/callback` endpoint
5. Writes to a DB
6. Notifies the customer

That's 300+ lines of code, a Dockerfile, a CI pipeline, and a server to maintain. n8n does the same with 4 visual workflows you can modify in-browser.

### 1.3 Installing n8n locally

```
cd docker/  
cp .env.example .env  
# edit .env  
docker compose up -d  
open http://localhost:5678
```

Log in with the basic-auth credentials from your `.env` .

## 1.4 The n8n UI

- **Workflows** — your automations (one per use case)
- **Executions** — history of every run (success / fail / running)
- **Credentials** — encrypted secrets (DB passwords, API keys)
- **Variables** — plaintext globals (shortcodes, URLs)

## 1.5 Your first workflow : "Hello Webhook"

1. Click **New Workflow**
2. Add a **Webhook** node → path = `hello` , method = `POST`
3. Add a **Set** node → add a field `message = Hello {{$json.body.name}}`
4. Click **Execute Workflow**, then POST to the test URL

```
curl -X POST http://localhost:5678/webhook-test/hello -d '{"name":"Titus"}' -H 'Content-Type: application/json'
```

Congratulations — you just built your first n8n API. The M-Pesa workflow is the same pattern, scaled up.

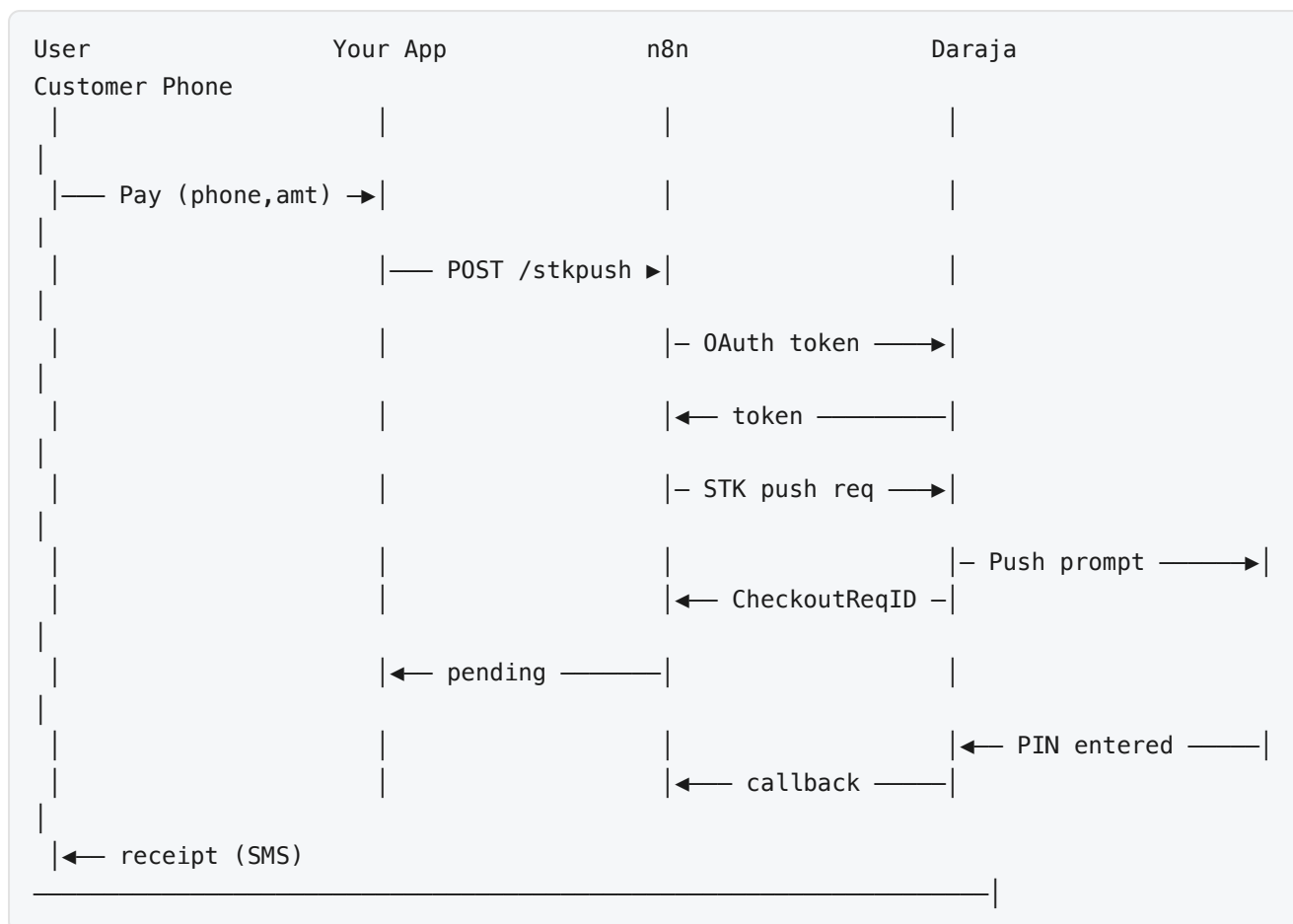
---

## Module 2 : Understanding the Daraja API

---

See `daraja-cheatsheet.md` for the one-page reference. Key points:

- **Sandbox vs Production** — different base URLs, different credentials. Switch via `MPESA_ENV` .
- **OAuth lifecycle** — Basic Auth with Consumer Key/Secret → Bearer token (3599s TTL).  
Cache it.
- **STK Push sequence:**



## Module 3 : OAuth in n8n

Import `workflows/01-mpesa-auth.json` . Walkthrough:

1. **Trigger** — "When called by another workflow" makes this a reusable sub-workflow.
2. **Code node — Build Basic Auth** — reads env vars and base64-encodes `key:secret` .
3. **HTTP Request** — GET `/oauth/v1/generate` with `Authorization: Basic ...` .
4. **Code node — Return Token** — returns `{ access_token, expires_in, expires_at }` .

### Caching strategy (optional upgrade)

Store the token in n8n's **Workflow Static Data**:

```

const data = $getWorkflowStaticData('global');
if (data.token && new Date(data.expires_at) > new Date(Date.now() + 60_000)) {
  return [{ json: { access_token: data.token, cached: true } }];
}

```

Only hit `/oauth/v1/generate` on cache miss. Saves ~200ms per transaction and avoids rate limits.

---

## Module 4 : Building the STK Push Workflow

---

Import `workflows/02-stk-push.json` . The flow:

1. **Webhook** receives `{ phone, amount, reference, description }`
2. **Validate Input** normalises phone & checks amount
3. **Execute Workflow** → **01-mpesa-auth** gets a Bearer token
4. **Build STK Payload** computes timestamp + password
5. **HTTP Request** POSTs to `/mpesa/stkpush/v1/processrequest`
6. **Postgres** saves a pending row
7. **Build API Response** returns the result to the caller

### Test it

```
curl -X POST http://localhost:5678/webhook/mpesa/stk-push \
-H 'Content-Type: application/json' \
-d '{"phone":"254708374149","amount":1,"reference":"ORDER-1"}'
```

You should see the STK prompt on the test phone (sandbox: any Safaricom number returns a mock prompt).

---

## Module 5 : Handling the Callback

---

Import `workflows/03-stk-callback.json` .

**CRITICAL:** Respond `200 OK` **immediately**. Safaricom does not retry callbacks, but a slow response can cause them to mark your endpoint as unhealthy.

The workflow:

1. **Webhook** on `/mpesa/stk-callback` , response mode = **On Received** (returns ACK before processing)
2. **Parse Callback** flattens the nested `Body.stkCallback.CallbackMetadata.Item[]` into clean fields
3. **Update Transaction** sets `status` , `mpesa_receipt_number` , etc. by `CheckoutRequestID`

4. **IF branch** — on success → build confirmation; on failure → log

## Exposing locally with ngrok

```
docker compose --profile tunnel up -d
docker compose logs ngrok | grep 'url='
# -> https://xxxx-xxx.ngrok-free.app
```

Put that URL in `.env` as `MPESA_CALLBACK_BASE_URL` and restart n8n.

---

## Module 6 : Status Query & Reconciliation

---

Callbacks sometimes don't arrive (user ignored the prompt, network blip, your server was down). Workflow `04-reconciliation.json` fixes this.

- Runs every 5 minutes
- Finds `pending` transactions older than 2 minutes
- Calls `stkpushquery/v1/query` to get the real status
- Updates the DB

This single workflow is what separates "hobby demo" from "production payment system".

---

## Module 7 : Production Hardening

---

### Deploy to a VPS

```
ssh root@your-server
curl -fsSL https://raw.githubusercontent.com/<you>/n8n-mpesa-course/main/deploy/vps-
setup.sh \
  | bash -s -- api.yourdomain.co.ke you@email.com
```

The script installs Docker, Nginx, Let's Encrypt, and clones this repo.

### Security checklist

- Change `N8N_BASIC_AUTH_PASSWORD` and `N8N_ENCRYPTION_KEY`
- Firewall: only 22, 80, 443 open
- DB backups via `pg_dump` cron
- Rotate Daraja Consumer Secret quarterly

- Monitor failed-login attempts via `fail2ban`

## Rate limiting

Add this to `nginx.conf` :

```
limit_req_zone $binary_remote_addr zone=stk:10m rate=5r/s;
location /webhook/mpesa/stk-push { limit_req zone=stk burst=10; proxy_pass
http://127.0.0.1:5678; }
```

## Module 8 : Real-World Use Cases

Each JSON in `workflows/use-cases/` is a working template. Customise and deploy:

| File                           | Scenario                                  |
|--------------------------------|-------------------------------------------|
| <code>ecommerce.json</code>    | WooCommerce order → STK push → mark paid  |
| <code>saas-billing.json</code> | Monthly subscription with retries         |
| <code>school-fees.json</code>  | Google Form → STK → PDF receipt via email |
| <code>donations.json</code>    | Bubble.io form → STK → thank-you WhatsApp |
| <code>whatsapp-bot.json</code> | WhatsApp message triggers STK push        |

## Module 9 : Beyond STK Push

Teasers for a future course:

- **B2C** — paying salaries, refunds, winnings
- **C2B Register URL** — merchant till notifications
- **Account Balance** — automated float checks
- **Reversal** — refund within 7 days

Same patterns, different endpoints. The `01-mpesa-auth` workflow is reusable for all of them.



## Module 10 : Selling Your Skill (*Agency tier*)

---

### Pricing templates (KES)

| Package  | Price   | Deliverables                                            |
|----------|---------|---------------------------------------------------------|
| Starter  | 25,000  | Deploy n8n + STK push + callback for one shortcode      |
| Standard | 75,000  | + Reconciliation + DB + 1 integration (Sheets/WhatsApp) |
| Premium  | 200,000 | + Multi-tenant, dashboard, 3-month support              |

### Client onboarding checklist

1. Get their Daraja production credentials (NDA first)
2. Get domain + VPS access (or provision your own)
3. Deliver the 4 workflows + Postman collection
4. Train one of their devs for 2 hours
5. 30-day warranty

## Appendix : Troubleshooting

---

| Symptom                              | Likely cause                     | Fix                                                                  |
|--------------------------------------|----------------------------------|----------------------------------------------------------------------|
| Invalid Access Token                 | Expired or wrong env             | Re-run auth workflow; verify <code>MPESA_BASE_URL</code> matches env |
| Bad Request – Invalid Timestamp      | Server clock drift               | <code>timedatectl set-ntp true</code>                                |
| Callback never arrives               | URL not public / HTTPS invalid   | Test with <code>curl</code> from an external machine                 |
| The initiator information is invalid | Wrong Passkey                    | Copy-paste from Daraja portal exactly                                |
| STK push succeeds but no row in DB   | Postgres credential wrong in n8n | Re-select the credential in the workflow node                        |
| ResultCode 1037                      | User didn't enter PIN in 60s     | Reconciliation workflow will mark it timeout                         |

